



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4
Технології розроблення програмного забезпечення
«Вступ до паттернів проектування.»

Виконав:

Студент групи ІА-32

Костінський Макарій

Перевірив:

Мягкий Михайло Юрійович

Зміст

Теоретичні відомості.....	2
Хід роботи	2
Діаграма класів реалізації патерну «Ітератор»	2
Висновок:	5

Тема: Вступ до паттернів проектування.

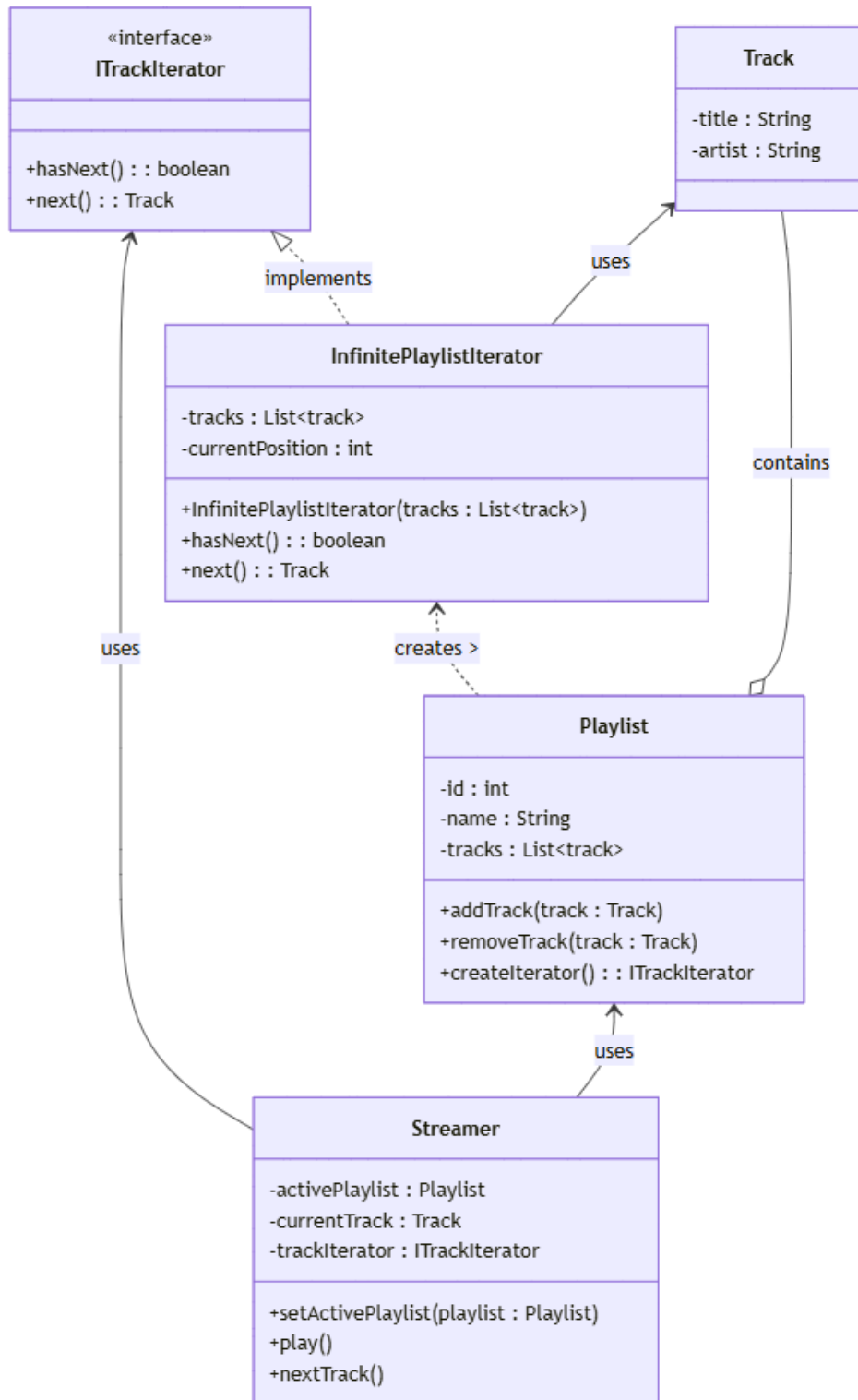
Мета: Вивчити структуру шаблонів «Singleton», «Iterator», «Proxy», «State», «Strategy» та навчитися застосовувати їх в реалізації програмної системи.

Теоретичні відомості

Шаблони проектування представляють собою формалізовані описи типових проблем проектування та їхніх вдалих рішень, що дозволяють використовувати перевірений досвід для побудови гнучких і зрозумілих архітектур програмного забезпечення. Патерн «Одинак» (Singleton) гарантує, що клас має лише один екземпляр, і надає глобальну точку доступу до нього, що є доцільним для контролю доступу до спільного ресурсу, хоча часто вважається «анти-шаблоном» через ускладнення тестування та наявність глобального стану. Шаблон «Ітератор» (Iterator) надає стандартний спосіб послідовного доступу до елементів колекції без розкриття її внутрішньої структури, виносячи логіку обходу в окремий об'єкт, що дозволяє реалізовувати різні алгоритми перебору даних незалежно від типу колекції. Патерн «Замісник» (Proxy) виступає як проміжний об'єкт або заглушка, що контролює доступ до іншого об'єкта, дозволяючи додавати додаткову логіку, таку як кешування, відкладена ініціалізація ("ліниве завантаження") або перевірка прав доступу, без зміни коду реального об'єкта. Шаблон «Стан» (State) дозволяє об'єкту кардинально змінювати свою поведінку при зміні його внутрішнього стану, інкапсулюючи код, специфічний для кожного стану, в окремі класи, що спрощує додавання нових станів і позбавляє код від громіздких умовних конструкцій. Патерн «Стратегія» (Strategy) визначає сімейство алгоритмів, інкапсулює кожен із них і робить їх взаємозамінними, що дозволяє клієнту динамічно вибирати необхідний алгоритм обробки даних під час виконання програми, відокремлюючи реалізацію алгоритму від коду, що його використовує.

Хід роботи

Діаграма класів реалізації патерну «Ітератор»



На діаграмі зображено структуру класів, що реалізують патерн Iterator для забезпечення навігації по списку треків у плейлісті.

ITrackIterator (Iterator): Інтерфейс, що визначає методи для обходу колекції: hasNext() для перевірки наявності елементів та next() для отримання поточного елемента.

```
import ua.kpi.iasa.onlineradio.models.Track;

public interface ITrackIterator { 5 usages 1 implementation

    boolean hasNext(); 1 usage 1 implementation

    Track next(); 1 usage 1 implementation
}
```

InfinitePlaylistIterator (Concrete Iterator): Конкретна реалізація ітератора. Він зберігає посилання на список треків та поточну позицію. Реалізує логіку "нескінченного" відтворення: при досягненні кінця списку перехід здійснюється на його початок.

```
import ...

public class InfinitePlaylistIterator implements ITrackIterator { 2 usages
    private final List<Track> tracks; 5 usages
    private int currentPosition = 0; 4 usages

    public InfinitePlaylistIterator(List<Track> tracks) { 1 usage
        this.tracks = tracks;
    }

    @Override 1 usage
    public boolean hasNext() {
        return !tracks.isEmpty();
    }

    @Override 1 usage
    public Track next() {
        if (tracks.isEmpty()) {
            return null;
        }
        Track track = tracks.get(currentPosition);
        currentPosition++;
        if (currentPosition >= tracks.size()) {
            currentPosition = 0;
        }
        return track;
    }
}
```

Playlist (Concrete Aggregate): Клас, що виконує роль колекції (агрегату). Він зберігає список об'єктів Track і реалізує фабричний метод createIterator(), який повертає новий екземпляр InfinitePlaylistIterator, пов'язаний з поточним списком треків.

```
public ITrackIterator createIterator() { 1 usage new *  
  
    return new InfinitePlaylistIterator(this.tracks);  
}
```

Streamer (Client): Клас-клієнт, який використовує об'єкти Playlist та ITrackIterator. Він не залежить від конкретної реалізації обходу (наприклад, індексів масиву), а делегує цю задачу інтерфейсу ITrackIterator. Це спрощує код клієнта і дозволяє легко змінювати алгоритм обходу (наприклад, на випадковий) без змін у самому Streamer.

```
public void nextTrack() { 2 usages 2 Makarii-IA-32 *  
    if (trackIterator == null) {  
        System.out.println("Неможливо перемкнути трек: плейлист не активний.");  
        return;  
    }  
    currentTrack = trackIterator.next();  
    if (currentTrack != null) {  
        System.out.println("🎧 Зараз в ефірі: " + currentTrack.getArtist() + " - " + currentTrack.getTitle());  
    } else {  
        System.out.println("Плейлист порожній.");  
    }  
}
```

Висновок:

Під час виконання лабораторної роботи засвоєно теоретичні відомості про патерни проєктування «Singleton», «Iterator», «Proxy», «State» та «Strategy», а також проаналізовано доцільність їх використання в різних сценаріях. Для предметної області «Online Radio Station» успішно реалізовано патерн «Ітератор», що дозволило створити механізм нескінченного відтворення плейлиста та відокремити алгоритм обходу треків від структури їх зберігання. Завдяки цьому отримано практичні навички модифікації архітектури програмного забезпечення з використанням шаблонів, що сприяє створенню гнучкого та розширюваного коду.